

Data Structures and Algorithms

Exercise 2: E-commerce Platform Search Function

1. Understanding Asymptotic Notation

What is Big O Notation?

Big O notation is used to measure the performance of an algorithm. It describes how the execution time increases as the input size increases.

Examples:

- $O(1) \rightarrow$ Constant Time
- $O(\log n) \rightarrow$ Logarithmic Time
- $O(n) \rightarrow$ Linear Time
- $O(n \log n) \rightarrow$ Linearithmic Time
- $O(n^2) \rightarrow$ Quadratic Time

Big O helps us compare algorithms and choose the most efficient one.

Search Cases

Best Case

- The element is found immediately.
- Linear Search: $O(1)$
- Binary Search: $O(1)$

Average Case

- The element is found somewhere in the middle.
- Linear Search: $O(n)$
- Binary Search: $O(\log n)$

Worst Case

- The element is the last element or not present.
- Linear Search: $O(n)$
- Binary Search: $O(\log n)$

2. Product Class

```
class Product
{
    int productId;
```

```
String productName;
```

```
String category;
```

```
Product(int productId, String productName, String category)
```

```
{
```

```
    this.productId = productId;
```

```
    this.productName = productName;
```

```
    this.category = category;
```

```
}
```

```
void display()
```

```
{
```

```
    System.out.println(productId + " " + productName + " " + category);
```

```
}
```

```
}
```

3. Linear Search

```
class LinearSearch
```

```
{
```

```
    static Product search(Product products[], int id)
```

```
    {
```

```
        for(int i = 0; i < products.length; i++)
```

```
        {
```

```
            if(products[i].productId == id)
```

```
            {
```

```
                return products[i];
```

```
            }
```

```
        }
```

```
        return null;
```

```
}
```

```
}
```

4. Binary Search

(Binary Search requires the array to be sorted by productId.)

```
class BinarySearch
```

```
{
```

```
    static Product search(Product products[], int id)
```

```
    {
```

```
        int low = 0;
```

```
        int high = products.length - 1;
```

```
        while(low <= high)
```

```
        {
```

```
            int mid = (low + high) / 2;
```

```
            if(products[mid].productId == id)
```

```
            {
```

```
                return products[mid];
```

```
            }
```

```
            else if(products[mid].productId < id)
```

```
            {
```

```
                low = mid + 1;
```

```
            }
```

```
        else
```

```
        {
```

```
            high = mid - 1;
```

```
        }
```

```
    }
```

```
        return null;
    }
}
```

5. Main Program

```
public class EcommerceSearch
{
    public static void main(String args[])
    {
        Product products[] =
        {
            new Product(101,"Laptop","Electronics"),
            new Product(102,"Shoes","Fashion"),
            new Product(103,"Mobile","Electronics"),
            new Product(104,"Watch","Accessories"),
            new Product(105,"Bag","Fashion")
        };

        int searchId = 104;

        System.out.println("Linear Search");

        Product result1 = LinearSearch.search(products, searchId);

        if(result1 != null)
        {
            result1.display();
        }
        else
        {
            System.out.println("Product Not Found");
        }
    }
}
```

```

    }

    System.out.println();

    System.out.println("Binary Search");

    Product result2 = BinarySearch.search(products, searchId);

    if(result2 != null)
    {
        result2.display();
    }
    else
    {
        System.out.println("Product Not Found");
    }
}
}

```

Output:

Linear Search

104 Watch Accessories

Binary Search

104 Watch Accessories

6. Time Complexity Analysis

Algorithm	Best Case	Average Case	Worst Case
Linear Search	$O(1)$	$O(n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$

7. Comparison

Linear Search

Advantages

- Works on unsorted data.
- Easy to implement.

Disadvantages

- Slow for large datasets.
- Checks elements one by one.

Binary Search

Advantages

- Very fast.
- Efficient for large datasets.

Disadvantages

- Requires the array to be sorted.

8. Which Algorithm is More Suitable?

Binary Search is more suitable for an e-commerce platform because:

- Product databases usually contain thousands or millions of products.
- Binary Search finds products much faster ($O(\log n)$).
- Products can be stored in sorted order based on productId, making Binary Search highly efficient.

Conclusion: For large product collections, **Binary Search** is preferred over **Linear Search** due to its significantly better performance.